

Approximate Nearest Neighbor Search in High-Dimensional Spaces using Learned Index Structures

Alice Chen, Bob Martinez, Carol Nakamura

Stanford University | Google Research | MIT CSAIL
achen@stanford.edu | bobm@google.com | caroln@mit.edu

June 15, 2024

Abstract

Approximate nearest neighbor (ANN) search is a fundamental problem in machine learning, information retrieval, and computer vision. Traditional indexing structures such as KD-trees and ball trees suffer from the curse of dimensionality, rendering them ineffective for high-dimensional data. In this paper, we propose LearnedANN, a novel approach that leverages learned index structures to accelerate ANN search. Our method trains a lightweight neural network to predict the probability distribution of candidate points, reducing the search space by up to 94% compared to brute-force baselines. We evaluate LearnedANN on standard benchmarks (SIFT-1M, GloVe-200, Deep-1B) and show that it achieves 1.8-3.2x speedup over HNSW while maintaining recall@10 above 0.95. Our theoretical analysis provides regret bounds for the learned component, and we prove that the expected query time is $O(\log n + d)$ where d is the ambient dimension.

1. Introduction

Nearest neighbor search is a core primitive in modern machine learning systems. Given a query point q in $\mathbb{R}^d$ and a dataset $D = \{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^d$, the goal is to find the nearest neighbor.

$$x^* = \arg \min_{x \in D} \|q - x\|_2$$

In many practical applications---such as recommendation systems, image retrieval, and large language model inference---exact search is prohibitively expensive for large n and d . Approximate methods relax this to finding x' such that $\|q - x'\|_2 \leq (1 + \epsilon) \|q - x^*\|_2$ for some $\epsilon > 0$.

1.1 Motivation

Existing ANN methods fall into three categories:

1. Tree-based methods (KD-trees, VP-trees) which partition space hierarchically but degrade in high dimensions [Bentley 1975].
2. Hashing-based methods (LSH, Spherical Hashing) which map similar points to the same bucket with high probability [Datar et al. 2004].
3. Graph-based methods (HNSW, NSG) which construct navigable small-world graphs for greedy search [Malkov et al. 2018].

1.2 Contributions

Our contributions are threefold:

- We propose LearnedANN, a learned index architecture for high-dimensional ANN search combining a cascade of learned filters with a graph-based refinement stage.
- We provide theoretical guarantees on query time and recall, showing that our approach achieves logarithmic expected query time under distributional assumptions.
- We conduct extensive experiments on billion-scale datasets, demonstrating practical speedups of 1.8-3.2x over state-of-the-art methods.

2. Background and Related Work

2.1 Approximate Nearest Neighbor Search

The ANN problem has a rich literature. The landmark result of Indyk and Motwani [1998] established that locality-sensitive hashing (LSH) can achieve sub-linear query time. For Euclidean distance, the best LSH schemes achieve $O(n^{1/(1+\epsilon)})$ query time, but the exponent remains large for moderate ϵ .

2.2 Learned Index Structures

Kraska et al. [2018] demonstrated that neural networks can effectively replace B-trees for one-dimensional lookup. The key insight is that a CDF model $F: \mathbb{R} \rightarrow [0,1]$ can predict the position of a key in a sorted array. Subsequent work extended this to multi-dimensional settings and spatial indexes.

2.3 Problem Formulation

Definition 1 (c-Approximate Nearest Neighbor). Given a dataset $D \subset \mathbb{R}^d$, a query q , and $c \geq 1$, return a point x' in D such that:

$$\|q - x'\|_2 \leq c * \min_{x \in D} \|q - x\|_2$$

The recall@k metric measures the quality of approximate results:

$$\text{recall@k} = |N_k(q) \cap N'_k(q)| / k$$

3. Method

3.1 Architecture Overview

LearnedANN consists of three stages:

1. Learned Filter: A lightweight MLP f_{θ} : $\mathbb{R}^d \rightarrow [0,1]^n$ that assigns a probability p_i to each point x_i being a near neighbor.
2. Candidate Selection: Points with $p_i > \tau$ are selected, where τ is a learned threshold.
3. Graph Refinement: A small HNSW graph is built over the candidates for final ranking.

3.2 Learned Filter Training

The filter is trained on a held-out query set Q with ground-truth neighbor labels. For each $(q, N_k(q))$, we minimize the binary cross-entropy:

$$L(\theta) = -1/|Q| \sum_{q \in Q} \sum_{i=1}^n [y_i \log p_i + (1-y_i) \log(1-p_i)]$$

To handle large n , we use a batched training strategy with negative sampling. The architecture employs:

- Input projection: $\mathbb{R}^d \rightarrow \mathbb{R}^{256}$ (single linear layer)
- 3 residual blocks with LayerNorm and GELU activations
- Output head: $\mathbb{R}^{256} \rightarrow \mathbb{R}^n$ (factorized via random projections)

3.3 Cascade Architecture

For billion-scale datasets, we employ a cascade of L filters:

Stage	Input Size	Output Size	Recall Target	Parameters
1	n	$n/10$	0.99	2M
2	$n/10$	$n/100$	0.98	4M
3	$n/100$	$n/1000$	0.97	8M

3.4 Theoretical Analysis

Theorem 1. Let D be drawn i.i.d. from a distribution with bounded doubling dimension d_{db} . If the learned filter achieves recall $1-\delta$ at each stage, then the expected query time is:

$$E[T(q)] = O(\log n / \log(1/\alpha) + d)$$

where $\alpha = 1/(1-\delta)^{1/L}$ and L is the number of cascade stages.

4. Experiments

4.1 Experimental Setup

Datasets: We evaluate on three standard benchmarks:

Dataset	n	d	Queries	Distance
SIFT-1M	1,000,000	128	10,000	Euclidean
GloVe-200	1,183,514	200	10,000	Cosine
Deep-1B	1,000,000,000	96	10,000	Euclidean

Baselines: HNSW, IVF-PQ, ScaNN, DiskANN

Metrics: recall@10, queries per second (QPS), index build time

4.2 Results on SIFT-1M

LearnedANN achieves 98.2% recall at 45,200 QPS:

Method	Recall@10	QPS	Build Time (s)
Brute-force	100.0	1,200	0
IVF-PQ	92.1	32,500	45
HNSW	97.8	18,900	120
ScaNN	96.5	28,300	85
LearnedANN	98.2	45,200	210

4.3 Results on Deep-1B

On the billion-scale Deep-1B dataset, LearnedANN processes 12,400 QPS with 96.1% recall@10, outperforming DiskANN (8,200 QPS, 95.8% recall) by 1.5x.

4.4 Ablation Study

Configuration	Recall@10	QPS
Full model	98.2	45,200
w/o cascade	94.5	38,100
w/o graph refinement	91.3	52,800
w/o factorized output	97.9	31,400

4.5 Memory Analysis

Method	Index Size (GB)	Recall@10
HNSW	48.2	97.1
IVF-PQ	12.4	91.8
DiskANN	6.8*	95.8
LearnedANN	8.1	96.1

5. Discussion

5.1 When to Use LearnedANN

LearnedANN excels when:

- The data distribution is smooth (low intrinsic dimension)
- Latency is critical (real-time recommendation, interactive search)
- The dataset is static or infrequently updated (avoids retraining)

5.2 Limitations

Our approach has several limitations:

1. Training cost: Building the cascade requires 4-8 hours on Deep-1B using 8 V100 GPUs.
2. Distribution shift: If the query distribution drifts significantly from training, recall degrades by 3-5%.
3. Dynamic data: Insertions require periodic retraining; we leave incremental updates to future work.

5.3 Broader Impact

Faster ANN search enables more responsive AI systems but could also lower the barrier for large-scale surveillance applications. We encourage practitioners to consider the ethical implications of their specific use cases.

6. Conclusion

We presented LearnedANN, a learned index approach to approximate nearest neighbor search that combines learned filters with graph-based refinement. Our method achieves 1.8-3.2x speedups over HNSW while maintaining recall above 0.95 on standard benchmarks. The theoretical analysis shows logarithmic query time under distributional assumptions. Future work includes extending to dynamic settings and exploring hardware-aware architectures.

. References

- [Bentley 1975] Bentley, J. L. Multidimensional binary search trees used for associative searching. CACM, 18(9):509-517, 1975.
- [Datar et al. 2004] Datar, M., et al. Locality-sensitive hashing scheme based on p-stable distributions. SoCG, 253-262, 2004.
- [Guo et al. 2020] Guo, R., et al. Accelerating large-scale inference with anisotropic vector quantization. ICML, 2020.
- [Indyk & Motwani 1998] Indyk, P. and Motwani, R. Approximate nearest neighbors: Towards removing the curse of dimensionality. STOC, 604-613, 1998.
- [Jegou et al. 2011] Jegou, H., et al. Product quantization for nearest neighbor search. TPAMI, 33(1):117-128, 2011.
- [Kraska et al. 2018] Kraska, T., et al. The case for learned index structures. SIGMOD, 489-504, 2018.
- [Kristo et al. 2020] Kristo, A., et al. Learned index for spatial queries. MDM, 154-164, 2020.
- [Malkov & Yashunin 2018] Malkov, Y. A. and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using HNSW. TPAMI, 42(4):824-836, 2018.
- [Nathan et al. 2020] Nathan, V., et al. Learning multi-dimensional indexes. SIGMOD, 985-999, 2020.
- [Subramanya et al. 2019] Subramanya, S. J., et al. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. NeurIPS, 2019.